

single_push_collective_tag.sh

Purpose:

Stored within the lcls2 repository, this script creates tags for deployed production versions of the daq within a hutch. This allows accurate bookkeeping in regards to what versions are deployed at a specific date per hutch.

Usage:

The script takes in arguments in order to be used across both lcls2 and ami. That being said, there is only one copy of the script and it lives within the lcls2 repo.

- hutch_name: the name of the hutch, e.g., xpp, tmo.
 - This is used as part of the tag name
- root_dir: This is the directory that stores the production lcls2 repos, i.e., the daqs at each hutch.
- tag_repo_path: This is the path to the repo that is used solely for pushing new tags. There is one for lcls2 and for ami.
 - /sdf/group/lcls/ds/ana/sw/conda2/rel/lcls_tag_repo/lcls2
 - /sdf/group/lcls/ds/ana/sw/conda2/rel/ami_tag_repo/ami
- prefix: This is used to designate whether the script is being used for ami or lcls2.
 - Use “ami” for ami
 - Use “lcls” for lcls2

Script:

This does not go over all parts of the script, but it does cover important details:

```
# Fetch all commits to ensure we have all the hashes
echo -e "${BLUE}Fetching latest commits...${NC}"
git fetch --all
echo -e "${GREEN}Fetch complete${NC}"
echo ""
```

The git fetch --all is applied to the tag repo. This is to ensure we are pushing to an up-to-date repo. We don't use pull because that does a merge to the local copy. A fetch is lighter.

The loop:

```
for item in "$ROOT_DIR"/*; do
    # Skip if not a directory
    if [ ! -d "$item" ]; then
        continue
    fi
```

This loops through each item in the root directory arg.

```
# Check if it's a git repository
```

```

if [ -d "$item/.git" ]; then
    REPO_NAME=$(basename "$item")

    # Only process repos that start with the requested prefix
    if [[ "$REPO_NAME" != ${PREFIX}* ]]; then
        continue
    fi

```

Within the loop, it checks that the item is a git repo. It also checks that the repo uses the prefix. This is because in the root directory there is a mix of ami and lcls2 clones.

```

# Get the clone commit hash
cd "$item"
GIT_HASH=$(git reflog --grep-reflog=clone -n 1 --format='%H' 2>/dev/null)

```

If the item is a repo and uses the prefix, it will get the hash of the commit when it was cloned.

```

CLONE_TIME=$(git reflog --grep-reflog=clone -n 1 --format='%ct' 2>/dev/null)

if [ -z "$CLONE_TIME" ]; then
    echo -e "${YELLOW} Warning: Could not determine clone time${NC}"
    continue
fi

DATE_STR=$(date -d @"$CLONE_TIME" +%Y%m%d)
TAG_NAME="${HUTCH_NAME}-${DATE_STR}"
echo -e "${GREEN} Tag Name: ${TAG_NAME}${NC}"

```

The tag name uses the hutch name argument and the date when the repo was cloned.

```

cd "$TAG_REPO_PATH"

# Check if commit exists in tag repo
if ! git rev-parse "$GIT_HASH" >/dev/null 2>&1; then
    echo -e "${YELLOW} Warning: Commit ${GIT_HASH} not found in tag repo.
Skipping.${NC}"
else
    # Check if tag already exists locally in tag repo
    if git rev-parse "$TAG_NAME" >/dev/null 2>&1; then

```

```

        echo -e "${YELLOW} Tag '${TAG_NAME}' already exists in tag repo.
Skipping creation.${NC}"
    else
        echo -e "${GREEN} Creating tag ${TAG_NAME} in tag repo${NC}"
        git tag -a "${TAG_NAME}" "$GIT_HASH" -m "Tag for ${HUTCH_NAME}"
install

```

It will check to see that the hash actually exists in the history of the tag repo. It will then check to see if the tag already exists.

This repeats across each item in the path and ends with a single push.

branch.sh

Purpose:

The core idea is that the script will monitor the DAQ directories in each hutch. If a change is detected, it will create a branch and push to the LCLS2 repo.

Usage:

The script takes in arguments in order to be used across both lcls2 and ami. That being said, there is only one copy of the script and it lives within the lcls2 repo.

- hutch_name: the name of the hutch, e.g., xpp, tmo.
 - This is used as part of the tag name
- root_dir: This is the directory that stores the production lcls2 repos, i.e., the daqs at each hutch.
- Branch dir: This is the path to a single base git repository where changes will be applied. There is one for lcls2 and for ami.
 - /sdf/group/lcls/ds/ana/sw/conda2/rel/branch_repo_lcls2/lcls2
 - /sdf/group/lcls/ds/ana/sw/conda2/rel/branch_repo_ami/ami
- prefix: This is used to designate whether the script is being used for ami or lcls2.
 - Use “ami” for ami
 - Use “lcls” for lcls2

In terminal: `./branch_out.sh <hutch_name> <root_dir> <branch_dir> <prefix>`

Script:

This does not go over all parts of the script, but rather over sections people may get stumped on.

```
for candidate in "$ROOT_DIR"/*; do
    [ -d "$candidate" ] || continue

    REPO_NAME=$(basename "$candidate")
```

This is the main loop. The purpose is to loop over all the candidate items in the production directory we are monitoring to check whether (later in the loop) the item is a production clone of a repo.

```
# Get short git hash from source repo
cd "$MONITOR_REPO"
echo -e "${GREEN} REMOTE FOLDER ${NC}"
echo -e "${YELLOW}Move to monitored folder ${NC}"

GIT_HASH=$(git rev-parse --short HEAD)
BRANCH_NAME="${HUTCH_NAME}-${REPO_NAME}"

echo "GIT_HASH=${GIT_HASH}"
echo "BRANCH_NAME=${BRANCH_NAME}"
```

When we find a candidate that is a cloned production repo, then we get the current git hash and set the branch name. The branch name is the hutch plus the repo name. The repo name is usually "repo_date". We use the repo name in the branch name because there will be multiple copies of the same repo that could need a branch.

```

for file in $CHANGED_FILES; do
    if [ -f "$MONITOR_REPO/$file" ]; then
        mkdir -p "$(dirname "$file")"
        cp "$MONITOR_REPO/$file" "$file"
        git add "$file"
        echo ".. $file synced and added to git"
    else
        if [ -f "$file" ]; then
            echo ".. $file deleted in source"
            git rm "$file"
        else
            echo ".. $file doesn't exist in local repo. Delete not needed, skipping"
        fi
    fi
done

```

This will apply the changes.

Cron

You need to be on `psrel@sdfcron001`

g. The cron jobs themselves need to be run as psrel. The “psrel” user is needed so that the dev team can update as needed, rather than a single developer. The jobs are set to once a week. There are two, one for lcls2 and one for ami.

Note: “psrel” has git push access on lcls2 and ami if using ssh clone.

These two exist only for xpp, but will be expanded to the other hutches when then migrate to sdf.

- # ---- LCLS2 TAG ----
- 0 2 * * 0
 /sdf/group/lcls/ds/ana/sw/conda2/rel/tag_repo_lcls2/lcls2/scripts/tag_scripts/single_push_collective_tag.sh xpp /sdf/group/lcls/ds/ana/sw/conda2/rel/xpp
 /sdf/group/lcls/ds/ana/sw/conda2/rel/tag_repo_lcls2/lcls2 lcls >
 /sdf/group/lcls/ds/ana/sw/conda2/rel/cron_logs/lcls2_tag/tag_job.log 2>&1 || echo "Tag job FAILED on \$(hostname) at \$(date). Check log." | mail -s "Tag Job Failure" mavaylon@slac.stanford.edu
-
- # ---- AMI TAG ----
- 0 2 * * 0
 /sdf/group/lcls/ds/ana/sw/conda2/rel/tag_repo_lcls2/lcls2/scripts/tag_scripts/single_push_collective_tag.sh xpp /sdf/group/lcls/ds/ana/sw/conda2/rel/xpp

```
/sdf/group/lcls/ds/ana/sw/conda2/rel/tag_repo_ami/ami ami >  
/sdf/group/lcls/ds/ana/sw/conda2/rel/cron_logs/ami_tag/tag_job.log 2>&1 || echo "AMI  
Tag job FAILED on $(hostname) at $(date). Check log." | mail -s "AMI Tag Job Failure"  
mavaylon@slac.stanford.edu
```

-

- # ---- LCLS2 BRANCH ----

- 0 2 * * *

```
/sdf/group/lcls/ds/ana/sw/conda2/rel/xpp/branch_lcls2/scripts/tag_scripts/branch_out.sh  
xpp /sdf/group/lcls/ds/ana/sw/conda2/rel/xpp  
/sdf/group/lcls/ds/ana/sw/conda2/rel/branch_repo_lcls2/lcls2 lcls >  
/sdf/group/lcls/ds/ana/sw/conda2/rel/cron_logs/lcls2_branch/branch_job.log 2>&1 || echo  
"LCLS2 Branch job FAILED on $(hostname) at $(date). Check log." | mail -s "LCLS2  
Branch Job Failure" mavaylon@slac.stanford.edu
```

-

- # ---- AMI BRANCH ----

- 0 2 * * *

```
/sdf/group/lcls/ds/ana/sw/conda2/rel/xpp/branch_lcls2/scripts/tag_scripts/branch_out.sh  
xpp /sdf/group/lcls/ds/ana/sw/conda2/rel/xpp  
/sdf/group/lcls/ds/ana/sw/conda2/rel/branch_repo_ami/ami ami >  
/sdf/group/lcls/ds/ana/sw/conda2/rel/cron_logs/ami_branch/branch_job.log 2>&1 || echo  
"AMI Branch job FAILED on $(hostname) at $(date). Check log." | mail -s "AMI Branch  
Job Failure" mavaylon@slac.stanford.edu
```